TransZKEval
Benchmark Construction Report

TransZKEval evaluates translation from imperative programming languages (IPLs) to zero-knowledge proof circuits. It contains 100 method-level tasks across eight domains, implemented independently in Python, Java, Rust, and Go, with one Circom reference implementation per task. These form 400 source-to-reference pairs and share 511 semantic test inputs.

This report follows the construction process: public code collection, functional analysis, domain and task allocation, expert requirement design, implementation in five languages, and validation. It presents the rationale for task selection together with the construction procedure.

1 Public Code Collection and Deduplication

We collected public Circom projects from GitHub, requesting the top 100 repositories ranked by stars using the query `language:circom fork:false`, and recursively collected their `.circom` files.

The local archive contains 6,407 Circom files. Template-level extraction produced 11,129 entries from 4,143 files across 99 repositories. Each entry retains its template name, code body, and source path to support functional analysis and retrieval.

Public projects reuse libraries, copy dependencies, and contain repeated implementations. We therefore deduplicated template bodies by content hash after trimming leading and trailing whitespace. This yielded 3,405 distinct template entries originating from 95 repositories. The deduplicated code forms a shared corpus for analysis and retrieval.

2 Code Sampling and Functional Analysis

For the initial functional analysis, we randomly sampled 346 templates without replacement from the 3,405 distinct entries. We used a 95% confidence level, a margin of error of 5 percentage points, and the conservative proportion p = 0.5. Applying the finite-population correction gives:

$$n_0 = \frac{1.96^2 \times 0.5(1-0.5)}{0.05^2} = 384.16, \qquad n = \left\lceil \frac{n_0}{1 + (n_0 - 1)/3405} \right\rceil = 346.$$

The saved sampling record uses seed 20261027. Functional analysis considered template names, source paths, and project purposes, with code-body inspection for selected entries to identify computational objectives, input-output relationships, and principal operations.

The initial analysis identified comparisons, range checks, conditional selection, integer arithmetic, bit operations, array access, and cryptographic computation, together with their combinations in model computation, financial applications, state verification, and interaction scenarios.

Building on the initial random sample, an application-focused review added AI and DeFi candidates from the corpus and organized entries from the other application categories, producing an application analysis set of 221 entries. The 346-template sample provides the initial functional inventory; the expanded application set provides the domain-specific analysis below. The analysis jointly records computational mechanisms (operations, access patterns, conditional relationships, and state dependencies) and application functions (the computational objectives and requirements served by those mechanisms).

The resulting primary functional categories are summarized below.

Primary functional category	Entries	Share
DeFi	92	41.6%
AI	42	19.0%
Cryptographic arithmetic and proof components	36	16.3%
Data structures and algorithms	27	12.2%
Blockchain and governance rules	7	3.2%
Statistics	5	2.3%
Games and interaction	3	1.4%
Other	9	4.1%
Total	**221**	**100%**

Table 1: Primary functional categories in the application analysis set (221 entries).

Entries with multiple functions receive a primary category based on their principal computational objective, together with supplementary mechanism labels. Signatures, commitments, and cryptographic verification are grouped under cryptographic arithmetic and proof components. Field extraction, array access, and queries are grouped under data structures and algorithms. Numerical aggregation and normalization are grouped under statistics. The remaining application checks, configuration components, and composite rules are grouped under Other.

This analysis connects application requirements with concrete computational mechanisms and provides a basis for domain selection and task design. The accompanying analysis records list the sampled templates and the application entries underlying these counts.

3 Establishing Domain Coverage

We combined the code analysis with the core functional modules and typical engineering scenarios of ZKP systems to establish seven application and computational domains.

Artificial intelligence covers feature combination, numerical transformation, selection, and information updates in model computation. Model components and supporting numerical operations provide design cues for different operators, computational paths, and intermediate dependencies.

DeFi covers financial formulas, conditional execution, and asset-state changes. Its code illustrates the combined requirements of fixed-point arithmetic, rounding, application thresholds, and execution order.

Cryptography covers modular, bit-level, and multistep algebraic relationships, emphasizing the preservation of numerical representations, operation ranges, and intermediate dependencies during circuit translation.

Blockchain covers ledger data and on-chain rules, emphasizing consistency among encoding, state verification, and state transitions.

Data structures and algorithms cover data access and storage updates, emphasizing arrays, indices, intermediate results, and dependencies across computational steps.

Statistics covers numerical analysis and data aggregation, emphasizing relationships among multiple inputs and how aggregation, ratios, scaling, and recurrence determine outputs.

Games and interaction covers verifiable state transitions and fairness, emphasizing relationships among rules, participant states, and interaction outcomes.

These domains introduce different application requirements and computational structures. Experts further develop them into method-level tasks that can be implemented independently and evaluated with shared tests.

4 Domain Allocation and Task Counts

Domain allocation considers the relative presence of categories in the application analysis set, functional variation within each domain, and balanced overall coverage. The final benchmark contains 100 method-level tasks.

First, we include 20 common simple control-flow tasks to evaluate foundational translation abilities. They cover decisions, selection, boundary handling, and numerical updates, with combinations ranging from individual operations to multistep logic.

Second, DeFi and AI occupy relatively large portions of the application analysis set and contain multiple functional modules. We therefore allocate 15 tasks to each. DeFi emphasizes differences in financial formulas, conditions, and state updates; AI emphasizes differences in operators, numerical paths, and information dependencies. These larger allocations accommodate variation within the two domains.

For cryptography, blockchain, data structures and algorithms, statistics, and games and interaction, we allocate 10 tasks each. Although their observed frequencies differ, balanced allocations and task diversity provide stable evaluation coverage for distinct types of computation.

Domain	**Tasks**	**Role of the allocation**
Simple Control-Flow	20	Assess common arithmetic, bitwise logic, decisions, and numerical updates
Artificial Intelligence	15	Cover neural-network operators, numerical paths, and information updates
DeFi Applications	15	Cover financial formulas, conditional rules, and asset-state changes
Games & Interaction	10	Cover verifiable interaction states and participant relationships
Cryptography	10	Cover modular, bit-level, and algebraic relationships
Blockchain	10	Cover ledger data and on-chain rules
Data Structures	10	Cover data access, algorithms, and multistep state dependencies
Statistics	10	Cover data aggregation and numerical analysis
Total	**100**	

Table 2: Task allocation across the eight benchmark domains, matching Table I in the paper.

5 Expert Design and Adaptation of Task Requirements

After determining the domains and allocations, experts manually design individual tasks based on the functional modules observed in code analysis and common requirements within each domain. The design process includes requirement organization, candidate specification, coverage-based selection, scale adaptation, and specification review.

5.1 Organizing Domain Requirements and Computational Features

Experts first divide each domain into functional directions and record their principal computational features, including input organization, numerical operations, conditional relationships, state updates, and output forms.

This produces a mapping between domain requirements and computational mechanisms. Different requirements within one domain may emphasize array accumulation, piecewise decisions, ratio calculations, or sequential updates. The same mechanism may also serve different purposes across domains.

The mapping identifies the functions each domain should cover and the demands these functions place on program-to-circuit translation.

5.2 Developing Candidate Functional Specifications

Experts formulate candidate requirements for each functional direction, first defining the computational objective and then establishing the input-output relationship.

Each candidate specification describes the available inputs, the required result, the formulas, conditions, and update steps affecting that result, and the handling of boundary cases. Specifications use language-independent descriptions to provide a common basis for implementation.

Candidate requirements address both application meaning and computational structure, turning domain needs into executable and testable method-level computations.

5.3 Selecting Tasks by Coverage Differences

Candidate specifications are organized in a coverage matrix to compare functional and structural differences. Selection considers four aspects:

- Functional contribution: whether a candidate adds a domain requirement or input-output relationship.
- Mechanism contribution: whether it adds different operations, conditions, access patterns, or state dependencies.
- Structural variation: whether it represents differences such as single-step versus multistep computation, individual versus combined conditions, or scalar versus array processing.
- Implementation feasibility: whether it supports a clear integer interface and can be implemented and validated in all five languages.

For candidates with similar computational relationships, preference is given to specifications with clearer coverage and more distinct contributions. Requirements are added to address gaps in the coverage matrix, and the task set is finalized according to domain allocations.

The larger AI and DeFi allocations retain different functional directions and combinations of mechanisms within these domains. The other domains prioritize computational types with clear differences. Simple control-flow tasks organize common mechanisms and their combinations into a progression from individual operations to multistep logic.

5.4 Controlling Scale and Adapting to Model Capabilities

Tasks use method-level granularity, encapsulating the central computation in an independent program. Experts control program length, data size, and external dependencies so that tasks fit a consistent context and code-generation setting.

The adaptation procedure includes:

- Expressing external state as explicit inputs.
- Using fixed or bounded data sizes.
- Bounding iteration while preserving update order.
- Retaining key access patterns and intermediate dependencies.
- Standardizing integer inputs and outputs and defining fixed-point representations where needed.
- Specifying scaling, rounding, remainders, numerical ranges, and boundary behavior.

The adapted specification preserves the core formulas, conditional relationships, and state changes that determine the output and establishes an explicit input-output contract.

5.5 Reviewing and Finalizing Unified Specifications

Before finalization, specifications are checked for clear computational objectives, complete input-output definitions, explicit boundary behavior, and a distinct contribution to domain coverage.

Ambiguities are resolved before implementation. Each task receives a unified requirement description containing its domain, functional objective, parameter definitions, computational logic, numerical rules, boundary behavior, and adaptations. This description serves as the shared basis for implementation in five languages and for test design.

Public code and its functional labels organize the RAG retrieval corpus. Evaluation tasks are implemented around the expert-defined requirements, connecting retrieval material and evaluation needs at the domain and functional levels.

6 Implementation in Five Languages and Cross-Review

Each unified functional specification is implemented in Python, Java, Rust, and Go. The four source versions preserve the same algorithmic logic and computational semantics while following common coding practices in each language.

Source programs use single-file encapsulation and consistent naming conventions. Comments are removed to standardize the information available during evaluation. Inputs and outputs are restricted to integer types to support cross-language comparison and integration with finite-field circuits.

Following the review procedure described in the paper, two software engineers with at least three years of experience cross-review the source programs. They check compliance with the unified requirements and consistency in integer arithmetic, execution order, and boundary handling.

Two ZKP experts with at least three years of experience then implement the verified computational logic in Circom. They express execution through arithmetic gates, signals, and constraints while handling signal dependencies and finite-field arithmetic. Each reference circuit is reviewed by another expert to check semantic alignment with the source programs.

The resulting corpus contains 400 verified IPL implementations and 100 Circom reference implementations: 500 implementation files representing 100 semantic tasks and 400 source-to-reference pairs.

7 Shared Testing and Consistency Validation

The shared test suite contains 511 semantic input-output cases. Each case is replayed through language-specific scripts for all four IPL implementations, yielding 2,044 source-language test executions. Tests cover principal execution paths, conditional branches, numerical boundaries, index access, and state updates.

Outputs from the four source implementations are compared with shared expected results. Circom references undergo header and main-component normalization, compilation, input-signal mapping, witness generation, and output extraction. Outputs are checked against the shared expected values modulo the scalar field. The compiler also extracts the constraint count used for the size-based analysis in the paper.

We assess functional test adequacy on the Python implementations using `pytest-cov` for statement and branch coverage and `mutmut` for mutation testing. As reported in the paper, the test suite achieves 99.78% statement coverage, 99.47% branch coverage, and an 82.22% mutation score. Expert review additionally checks boundary handling across implementations. Discrepancies are resolved against the unified functional specification and the affected checks are repeated.

We additionally check all 100 reference circuits for under-constraint using ConsCS, Picus, and ZkFuzz. None is reported as insecure; each circuit is either judged secure or receives an inconclusive result from the tools. The recorded outcomes distinguish a secure verdict from an unknown result, timeout, or absence of a counterexample.

Task identifiers link the final domain allocation, functional specifications, four IPL implementations, the Circom reference, and shared tests. This report documents the construction process and selection rationale; the associated code, tests, and analysis records provide the executable and inspectable materials.